

Expressions and Data Types

Introduction to computer programming

Management and Artificial Intelligence



Why Python?

- A high-level language for beginners
- with a clean syntax, but...
- Industrial strength programming language used at thousands of companies (one of three official Google languages)
- Free, well documented, well supported

Recall We will use Python 3 (there are non-backwards compatible differences with respect to Python 2). Further, Python 2 is now unsupported.

Expressions and Statements

A (Python) program is composed by a sequence of:

- definitions (also expressions) which are evaluated
- statements (also commands) which are executed

In plain terms...

- expressions yield some results (e.g., `5 * 5`)
- statements instruct the interpreter to do something (e.g., print on screen, assign to variable)

Expressions and statements can either

- be directly written into a shell (interactive mode)
- be written into a text file and then loaded into the shell (script mode).

Interactive vs. Script Mode

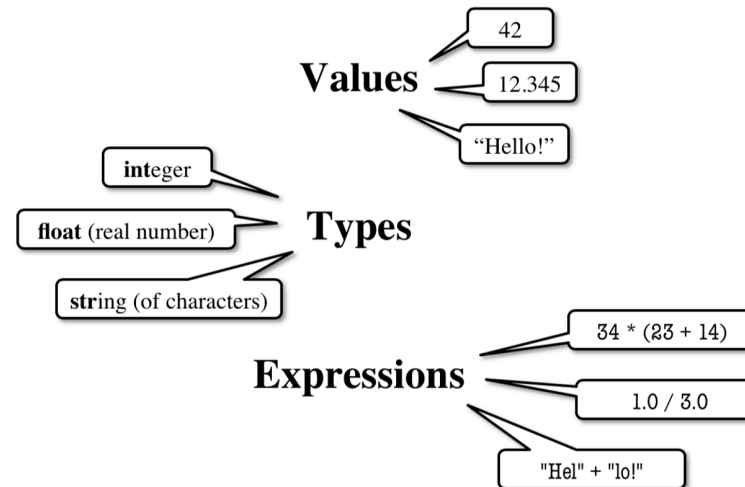
- Interactive Mode: perfectly fine for testing and learning.

Type your commands/expressions into the shell and Python will execute/evaluate those immediately:

>>> 5 * 5	←input
25	←output

- Script Mode: is what you normally do.
 - write the entire source code in a text file ending with the `.py` extension (say `myscript.py`)
 - `myscript.py` will be executed in its entirety by the Python interpreter

Programming in Python



- programs manipulate data objects
- each data object has its own type, which defines the type of operations that can be done with them
- objects can either be:
 - Scalar: cannot be subdivided
 - Non-scalar: have an inner structure that can be

Scalar Objects

- `int` : represents integers (e.g., `5`)
- `float` : represents real numbers (e.g., `3.14`)
- `bool` : represents Boolean values (`True` or `False`)
- `NoneType` : special with only one value (`None`)\

Hint: Use the ``type()`` function if you are not sure about the data type.

Examples:

```
>>> type(109)    ←input
```

```
<class 'int'>    ←output
```

```
>>> type(10.9)   ←input
```

```
<class 'float'>  ←output
```

Type `int()`

Values: $\dots, -3, -2, -1, 0, 1, 2, 3, \dots$

Integer literals look like a sequence of numbers with no periods nor commas.

Operations:

- `+`, `-`, `*`, `/`, `//` (float division), `%` (modulus, remainder), `**` (exponentiation)
- `-` (unary minus, to negate a number)
- `**` (exponentiation)

Examples:

<code>>>> 7/5</code>	←input
<code>1.4</code>	←output
<code>>>> 7//5</code>	← <code>//</code> is the floor division!
<code>1</code>	

Note In Python 3, operations between integers may not yield an integer.

Type `float()`

Values: some (approximated) range of $\mathbb{R}$

Note that:

- numbers with a "." are decimals (e.g., `2.0`)
- numbers with no decimal separator are integers (e.g., `2`)

Operations:

- `+`, `-`, `*`, `/`, `%` (modulus, remainder of division)
- `**` (exponentiation)
- `-` (unary, invert sign)

Operations between floats or between a float and an integer, always yield a float.

```
>>> 7.0/5.0
```

 ←input

```
1.4
```

 ←output

```
>>> 7.0//5
```

 ← `//` is the floor division!

```
1.0
```

Python stores floating point numbers as base 2 (binary) fractions:

$$\{\text{number}\} = \{\text{integer mantissa}\} \cdot 2^{-\{\text{exponent}\}}$$

For example: $0.125 = 1 \cdot 2^{-3} = 1 \cdot \frac{1}{2^3} = 1 \cdot \frac{1}{8}$.

Warning: Not all numbers can be represented in base 2 fractions. Python chooses the best approximation it can. The same also holds for some numbers in base 10 fraction: e.g., what about $1/3$?

Python may yield some approximation errors, that propagate as expressions go through.

Example:

>>> 0.1+0.2	←input
0.30000000000000004	←output
>>> 0.1+0.2+0.1+0.2	←input
0.6000000000000001	←output
>>> 0.1+0.2+0.1+0.2+0.1+0.2+0.1+0.2	←input
1.2000000000000002	←output

Type `bool`

Values:

`True` and `False` (capitalized!)

Operations:

$$\text{not } a = \begin{cases} \text{True} & \text{if } a \text{ is False} \\ \text{False} & \text{if } a \text{ is True} \end{cases}$$

$$a \text{ and } b = \begin{cases} \text{True} & \text{if both } b \text{ and } a \text{ are True} \\ \text{False} & \text{otherwise} \end{cases}$$

$$a \text{ or } b = \begin{cases} \text{True} & \text{if } a \text{ is True or } b \text{ is True} \\ \text{False} & \text{otherwise} \end{cases}$$

Booleans are the output of comparisons between items (e.g., `int` or `float`):

- Order: `i < j`, `i > j`, `i <= j`, `i >= j`
- (In)equality: `i != j`, `i == j`

Truth Table between two Boolean variables

a	b	not a	not b	a and b	a or b
True	True	False	False	True	True
False	False	True	True	False	False
True	False	False	True	False	True
False	True	True	False	False	True

Type Conversions

Switching between `float` and `int` can be performed via casting (explicit conversion):

- `float(2)` converts value `2` (integer) to `2.0` (floating point)
- `int(2.9)` converts value `2.9` (floating point) to `2` (integer)

Warning `int()` does not round to the nearest integer: rather, it truncates the decimal part. To round to the nearest integer, use `round()` instead.

Python supports widening automatically (implicit conversion):

$\underbrace{\text{bool} \rightarrow \text{int} \rightarrow \text{float}}_{\text{narrow to wide}}$

- Widening: Python does it automatically, if needed. Example:

`1/0.5=2.0` because Python casts `1` to `float`

- Narrowing: Python never does it automatically. Because narrowing leads to information loss!

Expressions

- Expressions are combinations of objects and operators
- an expression yields a value, which has its own type
- the simplest expression has the form:

`< object , operator , object >`

